AIOLA CPA, PLLC

Candidate Case Study

Full-Stack Software/AI Engineer

Confidential – For Candidate Use Only

OVERVIEW

Thank you for making it to this stage of the process. This case study is designed to give you a chance to show us how you think and how you build. There are no trick questions and no single right answer. We are looking for clarity of thought, good technical judgment, and an understanding of how security and compliance factor into your decisions.

Read through the full prompt before you start. If something is unclear, make a reasonable assumption, document it, and move forward.

Time to Complete	3 business days from receipt
Format	Written summary + working prototype or proof of concept
Submission	Email your deliverables to nick@aiolacpa.com
Presentation	You will walk us through your work live during the next interview (5-10 minutes)
Questions	Email nick@aiolacpa.com – reasonable clarifying questions are welcome

BACKGROUND

Aiola CPA serves real estate investors across the US. Our clients share sensitive financial information with us – tax returns, SSNs, bank information, entity structures – and we are responsible for handling that data carefully throughout the advisory, tax prep, and accounting process.

Today, we use a third-party client portal to collect tax organizers and documents from clients at the start of each engagement. The tool works, but it is expensive, not built for our specific workflow, and gives us limited ability to customize the experience. We want to replace it with something we own and control.

This is a real category of problem you would be solving in this role. The scenario below is not hypothetical – it reflects actual decisions we will be making.

THE PROMPT

You have been asked to design and build a lightweight internal client intake and document collection tool to replace our current third-party portal. This tool will be used at the start of each engagement to collect tax organizers, supporting documents, and sensitive data from our clients.

> *Your job is to design it, build a working prototype or proof of concept, and deliver it to the team.*

Part 1 – Architecture & Tech Stack

Walk us through how you would build this tool. Address the following:

- What tech stack would you use for the frontend, backend, and database, and why?
- How would the system be hosted and deployed?
- How would client-uploaded documents be stored and organized?
- What would authentication look like for both the client-facing side and the internal team side?

Part 2 – Security & Compliance

This is not an optional section. Security is foundational to how we build, and we will be evaluating how naturally it shows up in your thinking. Address the following:

- How would you ensure client data is encrypted in transit and at rest?
- How would you handle access controls so that only the right people can see the right data?
- IRC §7216 governs how client tax return information can be used and disclosed, including in software systems. How would you factor this into the design of this tool? You are not expected to know this statute in depth – a brief, honest answer about how you would approach learning and applying it is perfectly acceptable.
- What monitoring or alerting would you put in place?

Part 3 – The Build

Build a working prototype or proof of concept that demonstrates the core functionality. It does not need to be production-ready or fully featured. We are looking for real, working code, not a mockup or a slide deck.

At minimum, the prototype should demonstrate:

- A client-facing intake form or upload interface
- A basic backend that receives and stores the submitted data or files
- At least one security measure implemented in the code (not just described)

Go further if you have the time and want to show more. We will take note of it.

Part 4 – Rollout & Maintenance

Briefly address how you would approach getting this into production and keeping it running:

- What would the rollout process look like? How would you handle the transition from the current tool?
- How would you monitor the system after launch?
- What would break first, and how would you handle it?

WHAT TO SUBMIT

1. **Written summary** (PDF or Word) covering your architecture decisions, security approach, and rollout plan. Keep it concise – we read for substance, not length.
2. **Working code** Shared via GitHub or a zip file. Include a README with setup instructions so we can run it.
3. **Any assumptions you made** Document them clearly. We will not penalize reasonable assumptions, we will penalize undisclosed ones.

> *During next interview, be ready to walk through your code, explain your decisions, and answer follow-up questions. We are more interested in your reasoning than in a polished demo.*

HOW WE WILL EVALUATE THIS

We will score submissions across five areas. No single area is a disqualifier on its own – we are looking at the full picture.

Area	Weight	What We Are Looking For
Technical Execution	30%	Working code, clean structure, good decisions on stack and architecture
Security & Compliance	25%	Security built in from the start, honest engagement with §7216
Problem-Solving Approach	20%	How you handle ambiguity, document assumptions, and think through tradeoffs
Communication & Clarity	15%	How well you explain your decisions in writing and in the live walkthrough
Rollout & Maintenance Thinking	10%	Whether you have thought beyond the build itself

A NOTE FROM US

We know this takes real time and effort, and we appreciate it. This case study is the closest thing to a day-one work sample we can ask for, and it helps us make a better decision, for us and for you.

If you have questions along the way, reach out. We would rather you ask than make an assumption that sends you in the wrong direction.

Good luck, and we look forward to seeing what you build.